

Zomato Operations Analytics Dashboard Blueprint

Problem Statement

Zomato processes millions of orders daily, managing a complex network of users (customers), restaurants, and delivery partners. Currently, the data for restaurants and delivery partners exists in separate, disconnected datasets. Without a unified view, it is difficult to monitor real-time operations, track delivery efficiency, or analyze the geographical distribution of orders. There is a need for a centralized analytics dashboard with an interactive map to clean, merge, and visualize this data, allowing operators to filter and drill down into specific entities (users, restaurants, and delivery partners) for actionable insights.

Proposed Solution

We propose a Real-Time Operations Analytics Dashboard built on a unified, normalized database schema. The system will:

1. **Clean and Merge Data:** Ingest the existing disparate datasets for restaurants and delivery partners, and generate realistic sample data for users and orders to simulate a live environment.
2. **Centralized Database:** Use PostgreSQL to store the cleaned data in a normalized schema (Users, Restaurants, Delivery Partners, and Orders).
3. **Interactive Map Dashboard:** Build a web-based dashboard featuring a geospatial map. Operators can apply various filters (e.g., city, active status) and see the locations of restaurants and delivery partners.
4. **Entity Drill-Down:** Upon clicking or hovering over a map pin, the dashboard will display detailed information, such as the restaurant's name, the delivery partner's current data, and the associated user's location and order details.
5. **Modern ORM Integration:** Utilize Drizzle ORM, a lightweight and highly performant TypeScript Object-Relational Mapper, to handle database interactions safely and efficiently.

Tech Stack

- **Database:** PostgreSQL (for robust relational data storage and geospatial capabilities).
- **Backend ORM:** Drizzle ORM (for type-safe SQL query building and schema management in TypeScript).
- **Backend Framework:** Node.js with Express (to serve REST APIs for the dashboard).
- **Frontend Dashboard:** React or Next.js (for building the UI).
- **Mapping Library:** Leaflet.js or Mapbox GL (for rendering the interactive map and plotting coordinates).
- **Data Generation:** Python with the Faker library (to generate realistic sample user and order data).

Project Roadmap

Phase 1: Data Preparation and Generation

- Analyze Existing Data: Review the current restaurant and delivery partner datasets to identify common geographical attributes (e.g., latitude, longitude, city).
- Clean and Standardize: Write a Python script to clean the existing datasets, handling missing values and standardizing formats.
- Generate User Data: Use Python's Faker library to generate a realistic dataset for Users and their orders, ensuring the geographical coordinates align with the restaurant and delivery partner locations.

Phase 2: Database Schema and Setup

- Define the Schema: Design a Third Normal Form (3NF) PostgreSQL database schema with core entities: Users, Restaurants, DeliveryPartners, and Orders.
- Drizzle ORM Setup: Initialize Drizzle ORM in the Node.js backend. Define the schema in TypeScript using Drizzle's pgTable definitions.
- Data Insertion: Use Drizzle ORM or bulk SQL COPY commands to load the cleaned and generated datasets into the PostgreSQL database.

Phase 3: Backend API Development

- Set up Express Server: Initialize a Node.js/Express server to handle incoming requests.
- Develop Endpoints: Create REST APIs using Drizzle ORM queries to fetch data for the dashboard.
 - GET /api/map-data: Fetches coordinates and basic details for all entities.
 - GET /api/restaurant/:id: Fetches detailed restaurant and current order data.
 - GET /api/delivery-partner/:id: Fetches delivery partner status and assigned order details.

Phase 4: Frontend Dashboard Integration

- Build the UI: Create the dashboard layout with a sidebar for filters (e.g., status, location, order volume) and a main area for the map.
- Integrate the Map: Implement the mapping library to plot the geographical coordinates of the entities fetched from the API.
- Implement Interactions: Add click and hover event listeners to the map pins to trigger modals or tooltips displaying the detailed, drilled-down information for users, restaurants, and delivery partners.